Secure Messaging Web Application with ECDH & ChaCha20

ICS 344 – Information Security

Technical Report

Contents

1. Introduction and Objectives 3
1.1. What the System Does 3
1.2. Security Goals 3
1.3. Selected Buckets 3
2. Cryptographic Design (algorithms, modes, key sizes) 4
2.1. Algorithms Used 4
2.2. Modes 4
2.3. Libraries & Tools Used 7
3. GUI Design 8
3.1. Interface Screens 8
3.1.1. Login Screen 8
3.1.2. Messaging Interface 8
3.1.3. Attack Simulation Panels 9
3.2. Message Flow 11
3.3. Errors or Warnings 13
4. Workflow Diagram 16
4.1. System Architecture 19
4.2. Sender workflow 17
4.3. Sender workflow 17
5. Attack Simulations and Defenses 18
5.1. Dictionary Attack 18
5.2. Forged Signature Attack 19
5.3. Phishing Attack 20
5.4. Denial of Service (DoS) 21
6. Security Evaluation and Limitations. 23
6.1. Strengths 23
6.2. Limitations 23

1. Introduction and Objectives

1.1. What the System Does

This project implements a secure messaging web application using modern cryptographic primitives and security design principles. The application demonstrates confidentiality, integrity, authentication, key exchange, and attack resistance. The system uses a browser interface to send, receive, encrypt, decrypt, and validate secure messages in real time.

1.2. Security Goals

The project focuses on the following security goals:

- **Confidentiality:** Messages are encrypted using ChaCha20-Poly1305 AEAD mode.
- **Integrity & Authentication:** Each message is digitally signed, and signatures are verified during decryption to detect tampering.
- **Secure Key Exchange:** Elliptic Curve Diffie–Hellman (ECDH) is used to derive shared session keys between two users.
- **Attack Detection:** The system simulates real-world attacks including dictionary attack, forged signature, phishing, and denial of service (DoS).

1.3. Selected Buckets

- **Bucket 1:** ChaCha20-Poly1305 (authenticated cipher)
- **Bucket 2:** Elliptic Curve Diffie-Hellman (ECDH)
- **Bucket 3:**
  - Dictionary attack
  - Forged signature
  - Phishing
  - DoS
- **Bucket 4:** Web-based GUI (Flask Web App)
- **Bucket 5:** Python Implementation

2. Cryptographic Design

2.1. Algorithms Used

Algorithm	Purpose	Key size
ChaCha20-Poly1305	Authenticated Encryption	256 bits
ECDH (X25519)	Key Exchange	256 bits
Ed25519	Digital Signatures	256 bits

2.2. Modes

Encryption Flow

1. Sender obtains receiver's public ECDH key

```
# Derive shared session key using ECDH: sender's priv, receiver's pub
s_info = USERS[sender]
r_info = USERS[receiver]
session_key = derive_shared_key(s_info["ecdh_priv"], r_info["ecdh_pub"])
```

2. Sender derives symmetric encryption key

```
def derive_shared_key(private_key, peer_public_key) → bytes:  3 usages
    """
    Perform ECDH and derive a 256-bit symmetric key with HKDF(SHA-256).
    """
    shared_secret = private_key.exchange(ec.ECDH(), peer_public_key)
    derived_key = HKDF(
        algorithm=hashes.SHA256(),
        length=32,  # 256-bit
        salt=None,
        info=b"secure-messaging-ecdh"
    ).derive(shared_secret)
    return derived_key
```

3. Message encrypted using ChaCha20-Poly1305

4. Authentication tag automatically included

```
# Encrypt with ChaCha20-Poly1305
ciphertext, nonce = encrypt_with_key(session_key, plaintext_bytes, aad=None)
```

```
def encrypt_with_key(key: bytes, plaintext: bytes, aad: bytes | None = None):
    """
    Encrypt using ChaCha20-Poly1305 with a given 256-bit key.
    Returns (ciphertext, nonce).
    """
    nonce = os.urandom(12)  # 96-bit nonce
    chacha = ChaCha20Poly1305(key)
    ciphertext = chacha.encrypt(nonce, plaintext, aad)
    return ciphertext, nonce
```

5. Ciphertext is signed using Ed25519

```
def sign_data(private_key, data: bytes) → bytes:  2 usages
    """
    Sign arbitrary bytes using ECDSA (SHA-256).
    """
    return private_key.sign(data, ec.ECDSA(hashes.SHA256()))
```

6. JSON bundle transmitted to receiver

```
# Sign (ciphertext || nonce) with sender's signing private key
to_sign = ciphertext + nonce
signature = sign_data(s_info["sign_priv"], to_sign)

# Return bundle + metadata
return jsonify({
    "sender": sender,
    "receiver": receiver,
    "ciphertext": base64.b64encode(ciphertext).decode(),
    "nonce": base64.b64encode(nonce).decode(),
    "signature": base64.b64encode(signature).decode()
})
```

Decryption Flow

1. Receiver verifies signature

```python
# 1) verify signature using sender's signing public key
to_sign = ciphertext + nonce
if not verify_signature(s_info["sign_pub"], to_sign, signature):
    # integrity/authentication failure
    raise ValueError("Signature verification failed (message was tampered or forged).")
```

```python
def verify_signature(public_key, data: bytes, signature: bytes) -> bool:
    """
    Verify ECDSA signature; returns True if valid, False otherwise.
    """
    try:
        public_key.verify(signature, data, ec.ECDSA(hashes.SHA256()))
        return True
    except InvalidSignature:
        return False
```

2. Receiver derives same session key via ECDH

```python
# 2) derive the shared session key from ECDH
session_key = derive_shared_key(r_info["ecdh_priv"], s_info["ecdh_pub"])
```

3. ChaCha20 decrypts ciphertext

4. AEAD authentication tag is validated

5. Plaintext displayed

```python
# 3) decrypt with ChaCha20-Poly1305
try:
    plaintext_bytes = decrypt_with_key(session_key, ciphertext, nonce, aad=None)
except Exception:
    # Wrong key → almost always means "not the right recipient"
    raise ValueError("Decryption failed: wrong key or you are not the intended recipient.")

return jsonify({"plaintext": plaintext_bytes.decode("utf-8", errors="ignore")})
```

```python
def decrypt_with_key(key: bytes, ciphertext: bytes, nonce: bytes, aad: bytes | None = None):
    """
    Decrypt using ChaCha20-Poly1305 with a given 256-bit key.
    """
    chacha = ChaCha20Poly1305(key)
    return chacha.decrypt(nonce, ciphertext, aad)
```

2.3. Libraries & Tools Used

Cryptographic Library

- **cryptography.hazmat**
  - Provides secure high-level and low-level primitives.
  - Components used:
    - ChaCha20Poly1305 (authenticated encryption)
    - X25519PrivateKey and X25519PublicKey for ECDH
    - Ed25519PrivateKey and Ed25519PublicKey for signatures
  - Ensures safe key handling and secure randomness generation.

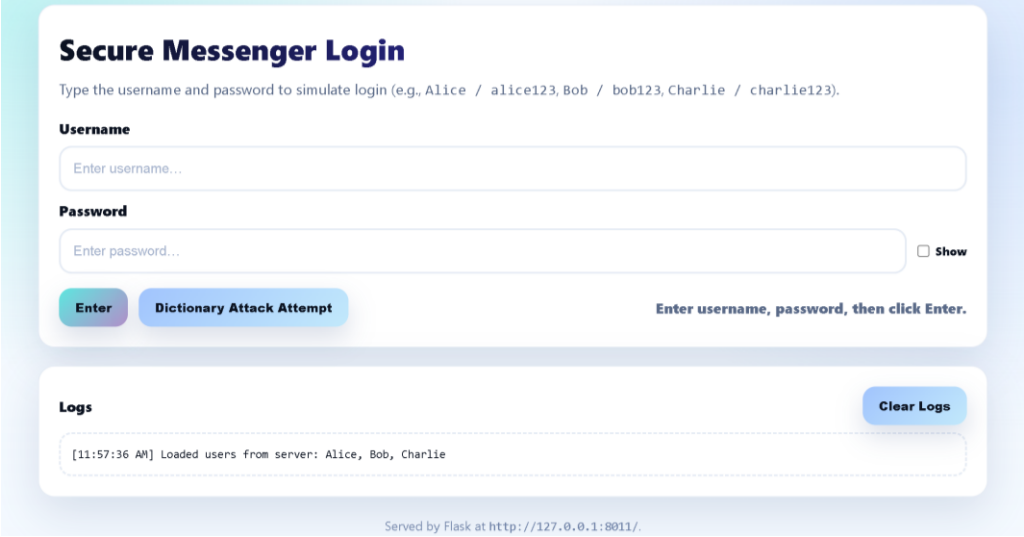
Utility Libraries

- **os.urandom**
  - Used for secure nonce and key generation.
- **base64**
  - Encodes ciphertext and signature into transport-safe text format.
- **json**
  - Used to process JSON-structured transport bundles.

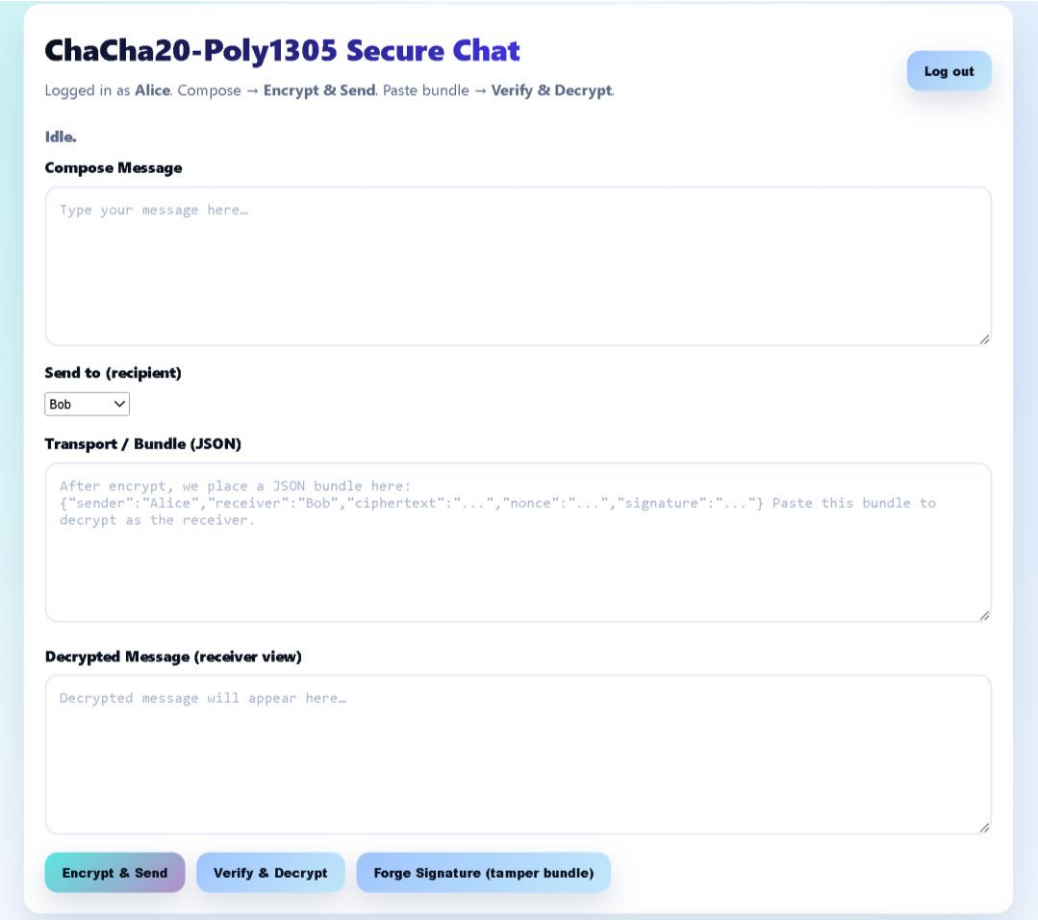
3. GUI Design

3.1. Interface Screens

3.1.1. Login Screen



3.1.2. Messaging Interface



3.1.3. Attack Simulation Panels

DoS / Flood Attack Simulation

6000 **Simulate DoS Flood**

DoS Simulation Stats:

Last run: **5000** requests in **151.82 ms** (avg **0.0304 ms** per request). Load level: **Normal.**

Recent runs:

#	Requests	Total (ms)	Avg (ms)
1	100	3.45	0.0345
2	2000	65.02	0.0325
3	5000	151.82	0.0304

Phishing Attack Simulation

From: **security@kfupm-it-support.com**
To: <your_email@kfupm.edu.sa>
Subject: **URGENT: Account Suspension Notice**

Dear user,

We detected unusual activity on your Secure Messenger account.
To avoid **immediate suspension,** please confirm your username and password
by logging in at the following link within the next 15 minutes:

http://secure-messenger-security-check.example.com/login

Show Fake Email **Click Suspicious Link**

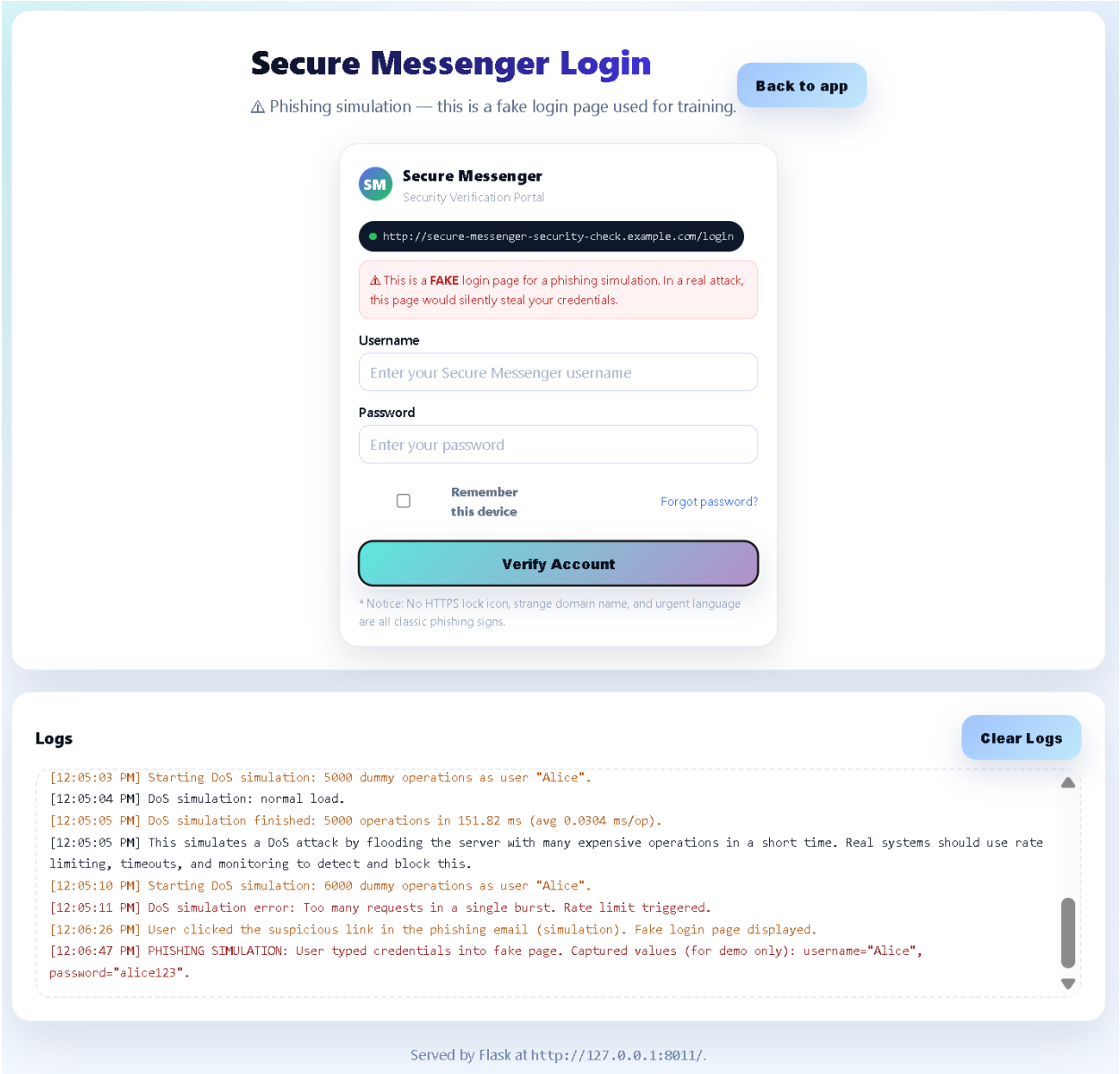
Logs

Clear Logs

[12:04:58 PM] DoS simulation finished: 2000 operations in 65.02 ms (avg 0.0325 ms/op).
[12:04:58 PM] This simulates a DoS attack by flooding the server with many expensive operations in a short time. Real systems should use rate limiting, timeouts, and monitoring to detect and block this.
[12:05:03 PM] Starting DoS simulation: 5000 dummy operations as user "Alice".
[12:05:04 PM] DoS simulation: normal load.
[12:05:05 PM] DoS simulation finished: 5000 operations in 151.82 ms (avg 0.0304 ms/op).
[12:05:05 PM] This simulates a DoS attack by flooding the server with many expensive operations in a short time. Real systems should use rate limiting, timeouts, and monitoring to detect and block this.
[12:05:10 PM] Starting DoS simulation: 6000 dummy operations as user "Alice".
[12:05:11 PM] DoS simulation error: Too many requests in a single burst. Rate limit triggered.

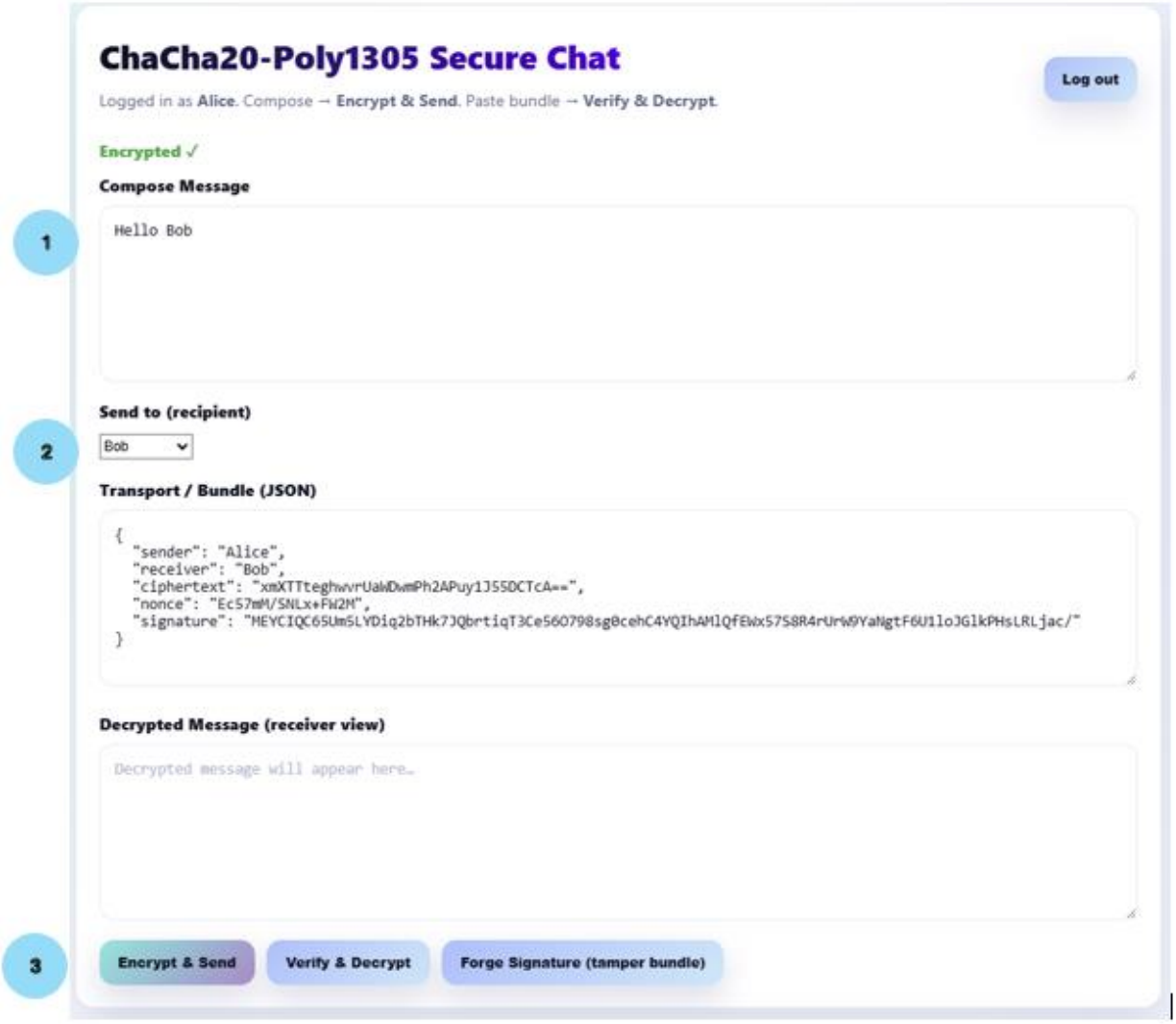
Served by Flask at http://127.0.0.1:8011/.

Fake Login page from phishing attack:

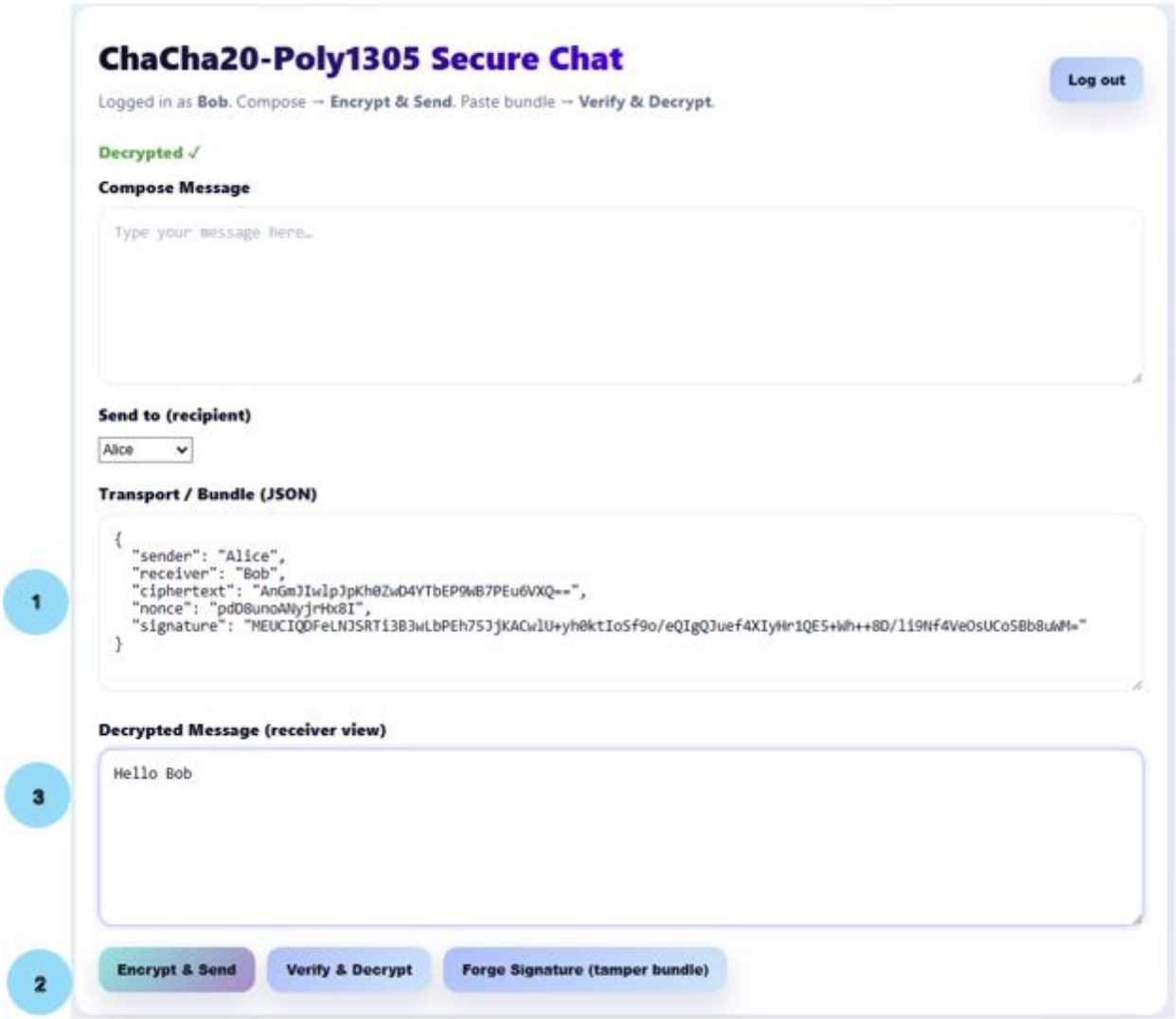


3.2. Message Flow

Plain text to Encryption

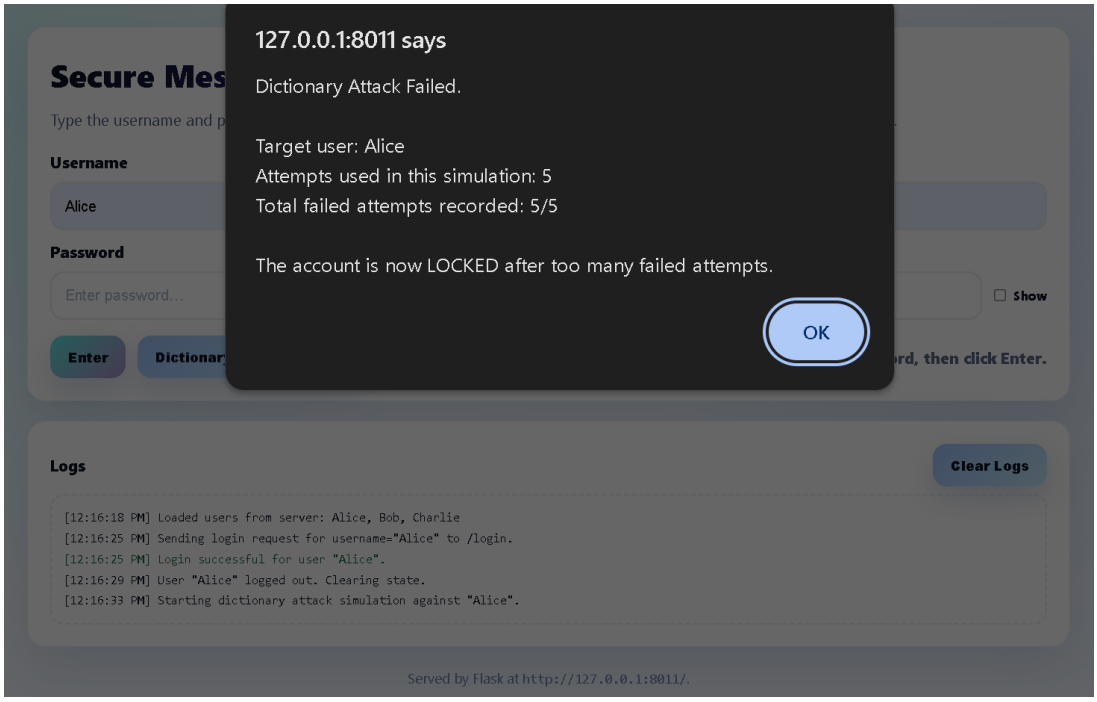


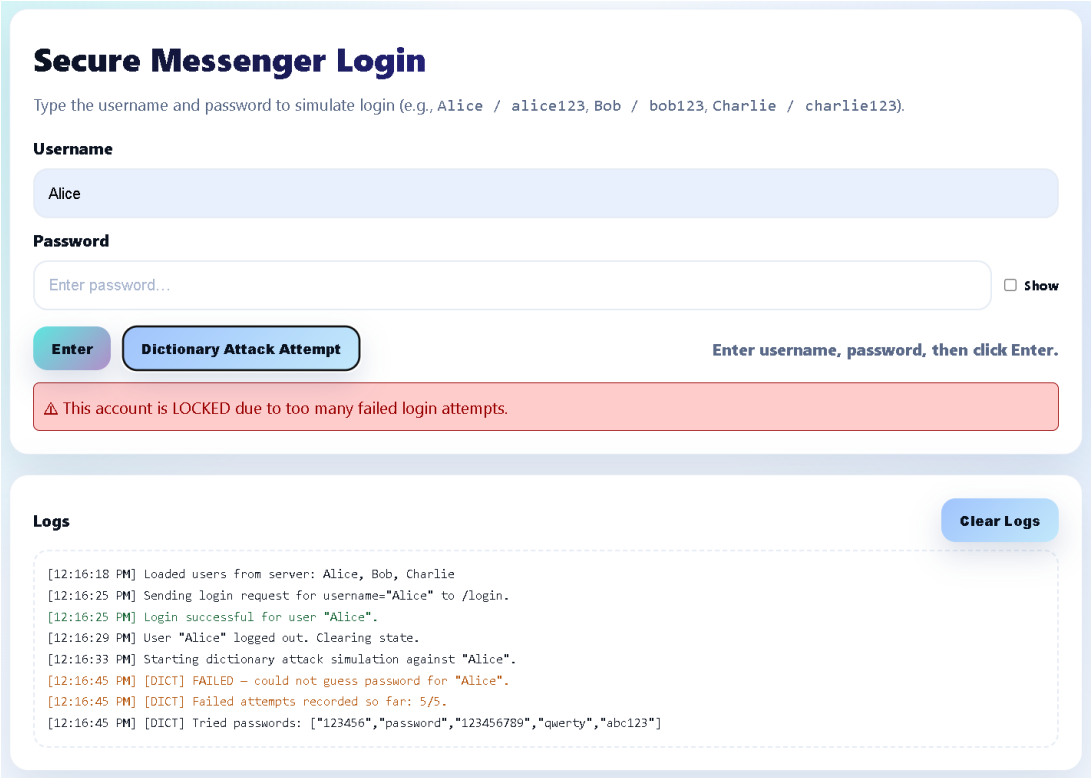
Verification & Decryption



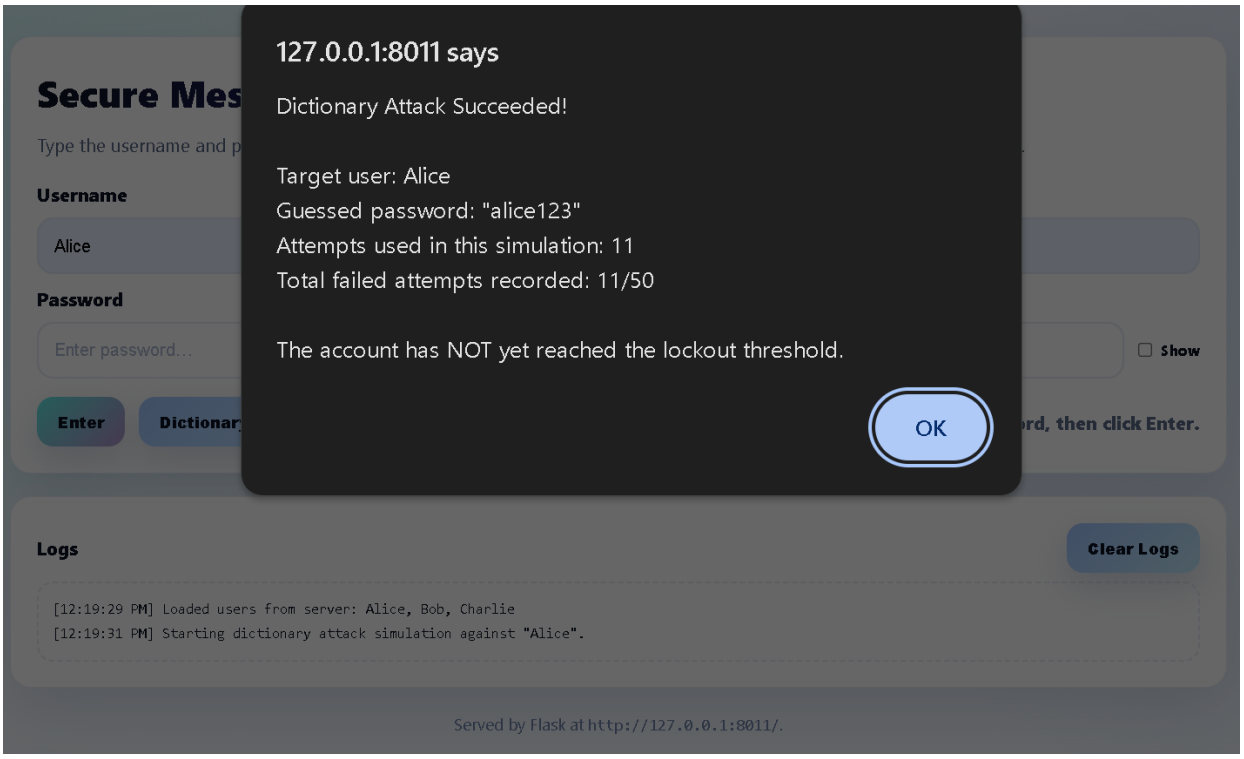
3.3. Errors or Warnings

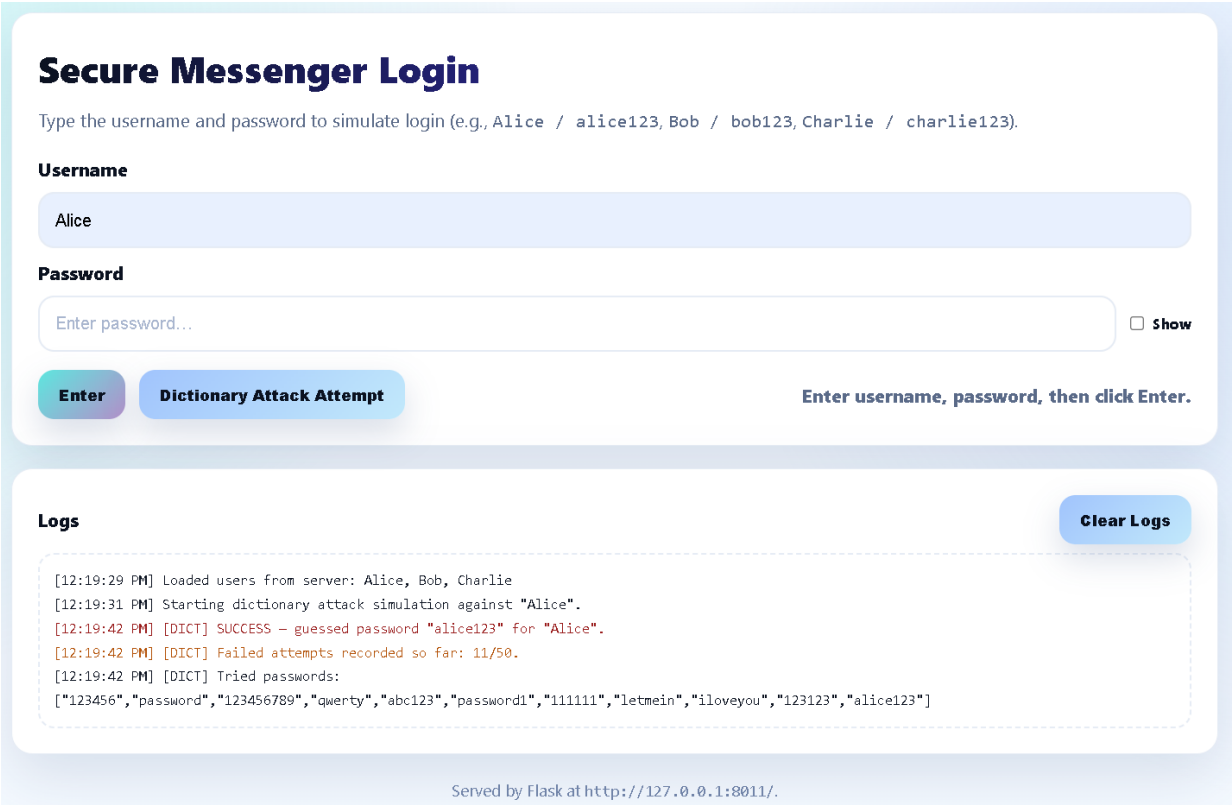
Dictionary Attack Failure:



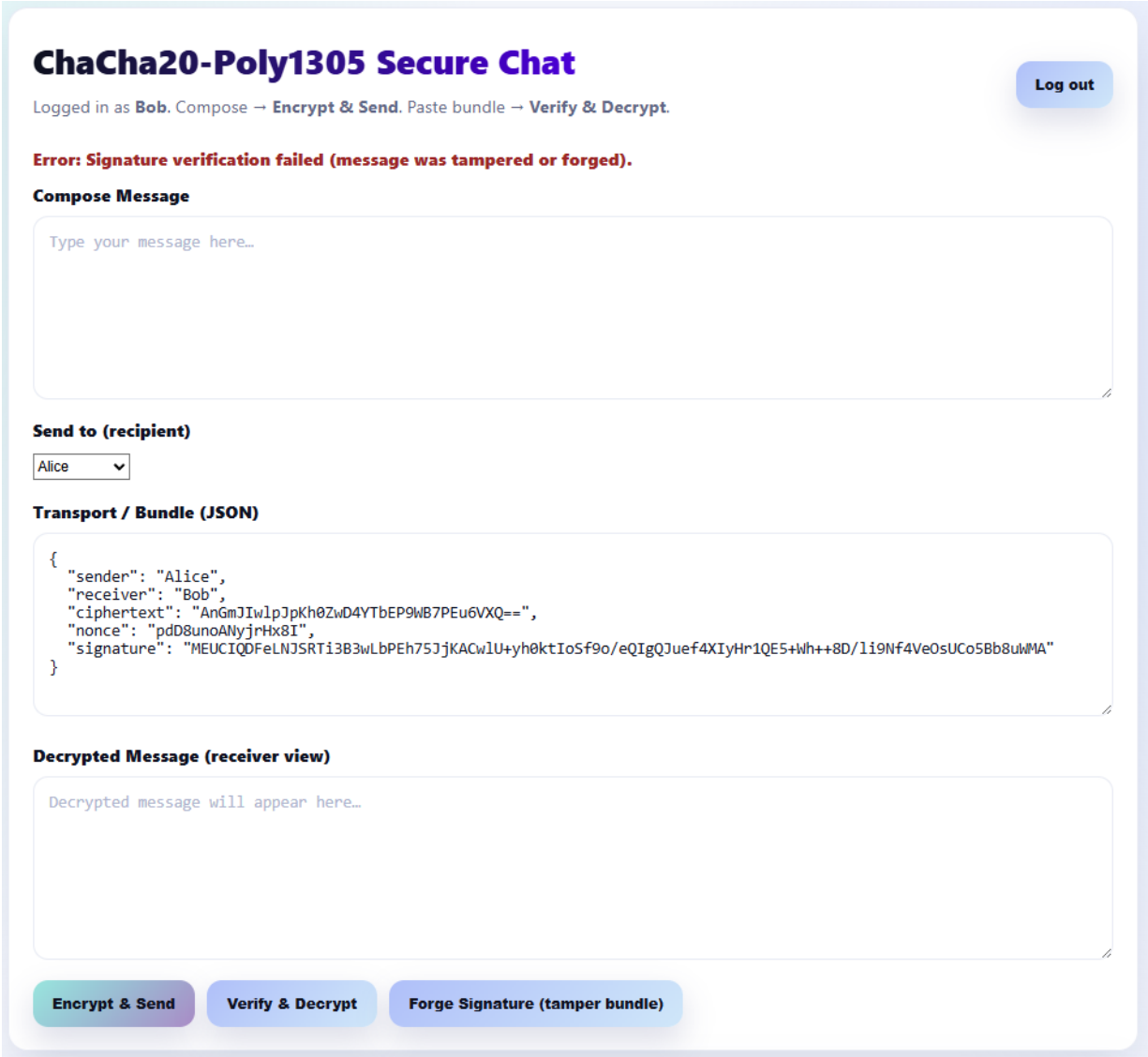


Dictionary Attack Success:





Forged Signature Detection:



Logs for DoS and Phishing Simulation:

4. Workflow Diagram.

4.1 System Architecture

Programming Language

- **Python 3.11**
  - Primary implementation language for backend logic and cryptographic operations.

Web Framework

- **Flask**
  - Used to build the interactive web-based user interface.
  - Routes HTTP requests and renders HTML templates.

Cryptographic Library

- **cryptography.hazmat**
  - Provides secure high-level and low-level primitives.
  - Components used:
    - ChaCha20Poly1305 (authenticated encryption)
    - X25519PrivateKey and X25519PublicKey for ECDH
    - Ed25519PrivateKey and Ed25519PublicKey for signatures
  - Ensures safe key handling and secure randomness generation.

Utility Libraries

- **os.urandom**
  - Used for secure nonce and key generation.
- **base64**
  - Encodes ciphertext and signature into transport-safe text format.
- **json**
  - Used to process JSON-structured transport bundles.

Frontend Technologies

- **HTML5 / CSS3**
  - For layout, forms, text input, card layout, dark/light UI elements.
- **JavaScript (Vanilla)**
  - UI interaction, event handling, button actions, logs, and input validation.

4.2. Sender workflow

1. **Compose plaintext**
   The sender types a normal text message from the GUI.

2. **Derive shared session key (ECDH)**
   The sender retrieves the receiver's public key and performs an Elliptic-Curve Diffie-Hellman calculation using X25519 to derive a shared secret.

3. **Encrypt using ChaCha20-Poly1305**
   The derived key, together with a random 96-bit nonce, is used to encrypt the plaintext message. The AEAD mode automatically generates an authentication tag internal to the ciphertext.

4. **Digitally sign encrypted ciphertext**
   The ciphertext is signed using the sender's Ed25519 private key to prove authenticity and prevent message forgery.

5. **Construct Transport Bundle (JSON)**
   The sender's name, receiver name, ciphertext, nonce, and signature are formatted into a JSON structure suitable for transport.

6. **Send bundle to receiver**
   The user manually transfers the JSON, simulating secure message delivery.

4.3. Sender workflow

1. **Receive JSON bundle**
   The receiver pastes the received Transport Bundle into the interface.

2. **Verify signature using Ed25519**
   The signature is validated using the sender's public key before attempting decryption. If verification fails, decryption is aborted.

3. **Derive shared ECDH session key**
   Using the receiver's private key and sender's public key, the receiver computes the same shared session key.

4. **Decrypt using ChaCha20-Poly1305**
   If signature validation succeeded, ChaCha20 decryption is performed using the shared session key and nonce.

5. **Display plaintext**
   The successfully decrypted message is rendered back into readable text within the GUI.

5. Attack Simulations and Defenses

5.1. Dictionary Attack

Definition:

Attackers attempt common passwords for a specific username until they find a match.

Execution

(part of dictionary attack function)

```
tried: List[str] = []
attempts = 0
found_password: Optional[str] = None

for candidate in wordlist:
    tried.append(candidate)
    attempts += 1

    if candidate == target_password:
        found_password = candidate
        break

    if max_attempts is not None and attempts ≥ max_attempts:
        break

success = found_password is not None
remaining = max(0, len(wordlist) - attempts)

if success:
    note = (
        f"Dictionary attack SUCCESS for user '{username}'. "
        f"Password guessed in {attempts} attempts."
    )
else:
    if max_attempts is not None and attempts ≥ max_attempts:
        note = (
            f"Dictionary attack stopped after reaching max_attempts={max_attempts} "
            f"for user '{username}'. Password not found in tested range."
        )
    else:
        note = (
            f"Dictionary attack FAILED for user '{username}'. "
            f"Password not present in provided dictionary."
        )
```

Detection

(in login route)

```python
# Check password
if info["password"] != password:
    info["failed_attempts"] = info.get("failed_attempts", 0) + 1
    remaining = max(0, MAX_FAILED_ATTEMPTS - info["failed_attempts"])

    # Lock if threshold reached
    if info["failed_attempts"] >= MAX_FAILED_ATTEMPTS:
        info["locked"] = True
        return jsonify({
            "error": "Account locked after too many failed login attempts.",
            "locked": True,
            "failed_attempts": info["failed_attempts"],
            "max_failed_attempts": MAX_FAILED_ATTEMPTS,
        }), 403

    return jsonify({
        "error": "Invalid password.",
        "locked": False,
        "failed_attempts": info["failed_attempts"],
        "remaining_attempts": remaining,
        "max_failed_attempts": MAX_FAILED_ATTEMPTS,
    }), 401
```

5.2. Forged Signature Attack

Definition

Attacker modifies ciphertext/signature to trick receiver.

Execution

(Forge Signature Helper)

```javascript
/* ---------- Base64 signature forge helper ---------- */

function forgeBase64Signature(sig) {  Show usages  Lena Ashqar
    const alphabet : string  = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";
    if (!sig || sig.length === 0) return sig;

    const lastChar = sig[sig.length - 1];
    const idx : number  = alphabet.indexOf(lastChar);
    if (idx === -1) return sig.slice(0, -1) + "A";

    const replacement = alphabet[(idx + 1) % alphabet.length];
    return sig.slice(0, -1) + replacement;
}
```

Detection

(Verify Signature method)

```python
def verify_signature(public_key, data: bytes, signature: bytes) -> bool:
    """
    Verify ECDSA signature; returns True if valid, False otherwise.
    """
    try:
        public_key.verify(signature, data, ec.ECDSA(hashes.SHA256()))
        return True
    except InvalidSignature:
        return False
```

5.3. Phishing Attack

Definition

Attacker tricks user into typing passwords on fake page.

Execution

Capturing user credentials from fake login page

```javascript
const btnSubmitPhish = $( id: "btnSubmitPhish");
if (btnSubmitPhish){
    btnSubmitPhish.addEventListener( type: "click",  listener: (ev : MouseEvent ) : void =>{
        ev.preventDefault();
        const u : any | string  = $( id: "phishUser") ? $( id: "phishUser").value.trim() : "";
        const p : any | string  = $( id: "phishPass") ? $( id: "phishPass").value.trim() : "";

        if (!u && !p){
            alert("In this simulation, enter some dummy username/password to see what happens.");
            return;
        }

        alert(
            "Phishing Simulation:\n\n" +
            "You just entered credentials into a FAKE login page.\n\n" +
            "In a real attack, the attacker would now have your username and password.\n" +
            "Always check the URL, HTTPS lock icon, and sender before entering credentials."
        );

        log(
             msg: `PHISHING SIMULATION: User typed credentials into fake page. ` +
            `Captured values (for demo only): username="${u}", password="${p}".`,
             type: "err"
        );
        setStatus( msg: "User fell for phishing (simulation only).",  cls: "err");

        if ($( id: "phishUser")) $( id: "phishUser").value = "";
        if ($( id: "phishPass")) $( id: "phishPass").value = "";
    });
}
```

Detection

No detection methods are available since this weakness is not the fault of the system rather it is the fault of the users. Implementing MFA may make this attack harder to execute but it is not guaranteed to protect against it completely. One way to mitigate against this attack is to train users against social engineering and phishing scenarios.

5.4. Denial of Service (DoS)

Definition

Attacker floods server with expensive operations.

Execution

```
def simulate_dos(  2 usages  Lena Ashqar *
    if count ≤ 0:
        count = 1

    # Simulated rate limiting: server refuses overly large bursts
    if count > max_ops:
        return DoSAttackResult(
            simulated_requests=0,
            blocked=True,
            duration_seconds=0.0,
            avg_seconds_per_request=0.0,
            note=(
                f"Requested {count} operations, which exceeds the allowed "
                f"burst limit of {max_ops}. Simulated rate limiting has blocked "
                "this DoS attempt."
            ),
            max_allowed=max_ops,
        )

    plaintext = b"A" * message_size  # dummy payload

    start = time.time()
    for _ in range(count):
        _ = simulate_expensive_encrypt(plaintext)
    end = time.time()

    duration = end - start
    avg = duration / count if count else 0.0

    return DoSAttackResult(
        simulated_requests=count,
        blocked=False,
        duration_seconds=duration,
        avg_seconds_per_request=avg,
        note=(
            "This simulates a DoS attack by flooding the server with many "
            "expensive operations in a short time. Real systems should use "
            "rate limiting, timeouts, and monitoring to detect and block this."
        ),
        max_allowed=max_ops,
    )
```

Detection

(Rate limit triggering)

```
if result.blocked:
    # Simulated rate limit hit
    return jsonify({
        "error": "Too many requests in a single burst. Rate limit triggered.",
        "simulated_requests": result.simulated_requests,
        "max_allowed": result.max_allowed,
        "blocked": True,
        "note": result.note,
    }), 429
```

6. Security Evaluation and Limitations.

6.1. Strengths

This project shows that secure messaging can be done in a clear and understandable way by combining modern cryptography with a simple user interface. Using ChaCha20-Poly1305, ECDH, and Ed25519 gives strong protection for confidentiality, authentication, and message integrity, and the interface helps users see each step happening in real time. The built-in simulations also make it easy to understand what happens when someone tries to guess passwords, tamper with messages, or launch phishing attacks, which makes the application useful not only as a secure messenger but also as a hands-on learning tool.

6.2. Limitations

Because this is a teaching project rather than a commercial system, it does not include a fully trusted public key infrastructure or secure identity verification. In real-world deployments, users would need a reliable way to confirm that a public key truly belongs to the person they think they are talking to. Also, since the system runs locally and uses manual copy-and-paste message transfer, it doesn’t reflect the complexity of large-scale secure messaging services. In the future, adding certificate validation, secure transport (like HTTPS), or automatic message delivery would make the system more realistic and more resistant to advanced attacks.